

## Lab Assignment 2: Smart City Traffic & Environmental Monitoring

**Scenario:** We are analyzing sensor data from different city zones to manage traffic congestion and air quality levels using Python, NumPy, and Pandas.

```
import pandas as pd
import numpy as np

# 1. Dataset Generation: High-Fidelity Sensor Data
data = {
    'Zone_ID': ['Z-01', 'Z-02', 'Z-03', 'Z-04', 'Z-05', 'Z-06'],
    'Area_Name': ['Downtown', 'Industrial Park', 'Suburban Mall', 'Tech Hub', 'Green Park', 'University'],
    'Traffic_Count': [450, 120, 310, 580, 45, 210],
    'CO2_Level_PPM': [410, 850, 430, 490, 320, 380],
    'Humidity_%': [45.5, 30.2, 55.0, 48.1, 72.4, 50.5],
    'Sensor_Status': ['Active', 'Active', 'Maintenance', 'Active', 'Active', 'Active']
}

city_df = pd.DataFrame(data)
print("--- Raw Smart City Sensor Data ---")
display(city_df)
```

--- Raw Smart City Sensor Data ---

|   | Zone_ID | Area_Name       | Traffic_Count | CO2_Level_PPM | Humidity_% | Sensor_Status |
|---|---------|-----------------|---------------|---------------|------------|---------------|
| 0 | Z-01    | Downtown        | 450           | 410           | 45.5       | Active        |
| 1 | Z-02    | Industrial Park | 120           | 850           | 30.2       | Active        |
| 2 | Z-03    | Suburban Mall   | 310           | 430           | 55.0       | Maintenance   |
| 3 | Z-04    | Tech Hub        | 580           | 490           | 48.1       | Active        |
| 4 | Z-05    | Green Park      | 45            | 320           | 72.4       | Active        |
| 5 | Z-06    | University      | 210           | 380           | 50.5       | Active        |

Next steps:

[Generate code with city\\_df](#)

[New interactive sheet](#)

```
# 2. NumPy Engineering: Sensor Normalization and Aggregation
traffic = city_df['Traffic_Count'].values
co2 = city_df['CO2_Level_PPM'].values
```

```
# Calculate normalized pollution index (CO2 relative to 400ppm baseline)
```



```
pollution_index = (co2 - 400) / 400

# Use NumPy for statistics
peak_traffic = np.max(traffic)
city_avg_co2 = np.mean(co2)

print(f"Peak Traffic recorded: {peak_traffic} vehicles")
print(f"City-wide Average CO2: {city_avg_co2:.2f} PPM")
city_df['Pollution_Index'] = pollution_index
```

Peak Traffic recorded: 580 vehicles  
City-wide Average CO2: 480.00 PPM

```
# 3. Pandas: Advanced Multi-Condition Filtering
# Identify zones that are 'At Risk' (High Traffic AND High CO2)
active_sensors = city_df[city_df['Sensor_Status'] == 'Active']
critical_zones = active_sensors[(active_sensors['Traffic_Count'] > 300) & (active_sensors

print("Zones Requiring Immediate Environmental Intervention:")
display(critical_zones[['Area_Name', 'Traffic_Count', 'CO2_Level_PPM']])
```

Zones Requiring Immediate Environmental Intervention:

|   | Area_Name | Traffic_Count | CO2_Level_PPM |  |
|---|-----------|---------------|---------------|-----------------------------------------------------------------------------------|
| 3 | Tech Hub  | 580           | 490           |                                                                                   |

```
# 4. Python Logic & Automation: Smart City Reporting System
def evaluate_zone_health(row):
    if row['CO2_Level_PPM'] > 600: return "DANGER: Hazardous Air Quality"
    elif row['Traffic_Count'] > 400: return "WARNING: Reroute Traffic"
    else: return "HEALTHY: Normal Parameters"

print("--- AUTOMATED CITY ADVISORY REPORT ---")
for i in range(len(city_df)):
    zone = city_df.iloc[i]
    report = evaluate_zone_health(zone)
    status_symbol = "🔴" if "DANGER" in report else "⚠️" if "WARNING" in report else "✅"
    print(f"{status_symbol} {zone['Area_Name']:<15} | Status: {report}")
```

```
--- AUTOMATED CITY ADVISORY REPORT ---
⚠️ Downtown | Status: WARNING: Reroute Traffic
🔴 Industrial Park | Status: DANGER: Hazardous Air Quality
✅ Suburban Mall | Status: HEALTHY: Normal Parameters
⚠️ Tech Hub | Status: WARNING: Reroute Traffic
✅ Green Park | Status: HEALTHY: Normal Parameters
✅ University Square | Status: HEALTHY: Normal Parameters
```

## ✓ 5. Advanced Data Operations

Below are detailed operations performed separately for better clarity.

```
# 5.1 Data Cleaning & Validation
# Check for missing values in the dataset
missing_values = df.isnull().sum()
```



```
print("Missing values per column:")
print(missing_values)

# Ensure all marks are within valid range (0-100)
invalid_marks = df[(df['Math'] < 0) | (df['Math'] > 100)]
print(f"\nNumber of invalid entries found: {len(invalid_marks)}")
```

Missing values per column:

```
Student_ID      0
Name            0
Math            0
Science         0
English         0
Average_Mark    0
Status          0
dtype: int64
```

Number of invalid entries found: 0

```
# 5.2 Detailed Subject Analysis (NumPy)
# Calculate Median and Standard Deviation for each subject separately
subjects = ['Math', 'Science', 'English']
for sub in subjects:
    sub_data = df[sub].values
    print(f"--- {sub} Statistics ---")
    print(f"Median: {np.median(sub_data)}")
    print(f"Std Dev: {np.std(sub_data):.2f}")
    print(f"Range: {np.ptp(sub_data)}\n")
```

--- Math Statistics ---

```
Median: 77.0
Std Dev: 21.56
Range: 62
```

--- Science Statistics ---

```
Median: 76.0
Std Dev: 19.66
Range: 56
```

--- English Statistics ---

```
Median: 76.0
Std Dev: 20.07
Range: 56
```

```
# 5.3 Specialized Filtering (Pandas)
# Find 'Top Performers' (Avg > 80 AND no subject below 70)
top_performers = df[(df['Average_Mark'] > 80) & (df['Math'] >= 70) & (df['Science'] >= 70) & (df['English'] >= 70)]

print("Top Performers with consistent scores:")
display(top_performers)
```



Top Performers with consistent scores:

|   | Student_ID | Name  | Math | Science | English | Average_Mark | Status      |  |
|---|------------|-------|------|---------|---------|--------------|-------------|-------------------------------------------------------------------------------------|
| 0 | 101        | Alice | 85   | 90      | 88      | 87.666667    | Distinction |  |
| 3 | 104        | David | 92   | 85      | 89      | 88.666667    | Distinction |                                                                                     |
| 6 | 107        | Grace | 88   | 94      | 91      | 91.000000    | Distinction |                                                                                     |

Next steps:

[Generate code with top\\_performers](#)

[New interactive sheet](#)

# 5.4 Modular Reporting Logic (Functions)

```
def generate_letter_grade(score):
    """Converts numerical average to a letter grade."""
    if score >= 90: return 'A+'
    elif score >= 80: return 'A'
    elif score >= 70: return 'B'
    elif score >= 60: return 'C'
    elif score >= 50: return 'D'
    else: return 'F'

# Add Grade column using the function
df['Grade'] = df['Average_Mark'].apply(generate_letter_grade)

print("Final Grade Distribution:")
print(df['Grade'].value_counts())
display(df[['Name', 'Average_Mark', 'Grade']])
```

Final Grade Distribution:

Grade

A 3  
F 2  
D 1  
A+ 1  
B 1

Name: count, dtype: int64

|   | Name    | Average_Mark | Grade |  |
|---|---------|--------------|-------|-------------------------------------------------------------------------------------|
| 0 | Alice   | 87.666667    | A     |                                                                                     |
| 1 | Bob     | 41.666667    | F     |                                                                                     |
| 2 | Charlie | 80.000000    | A     |                                                                                     |
| 3 | David   | 88.666667    | A     |                                                                                     |
| 4 | Eve     | 57.666667    | D     |                                                                                     |
| 5 | Frank   | 36.666667    | F     |                                                                                     |
| 6 | Grace   | 91.000000    | A+    |                                                                                     |
| 7 | Heidi   | 72.666667    | B     |                                                                                     |

✓ Version 2: Retail Sales & Inventory Management Analysis



This section uses a new dataset focused on store inventory, daily sales, and profit analysis.

```
import numpy as np
import pandas as pd

# 1. Create a Retail Dataset
inventory_data = {
    'Product_ID': ['P001', 'P002', 'P003', 'P004', 'P005', 'P006'],
    'Category': ['Electronics', 'Home', 'Electronics', 'Grocery', 'Grocery', 'Home'],
    'Cost_Price': [500, 20, 1200, 5, 8, 45],
    'Selling_Price': [650, 35, 1500, 7, 12, 60],
    'Stock_Level': [15, 120, 5, 300, 250, 10],
    'Daily_Units_Sold': [2, 15, 1, 45, 30, 8]
}

sales_df = pd.DataFrame(inventory_data)
print("--- Initial Inventory Data ---")
display(sales_df)
```

--- Initial Inventory Data ---

|   | Product_ID | Category    | Cost_Price | Selling_Price | Stock_Level | Daily_Units_Sold |
|---|------------|-------------|------------|---------------|-------------|------------------|
| 0 | P001       | Electronics | 500        | 650           | 15          | 2                |
| 1 | P002       | Home        | 20         | 35            | 120         | 15               |
| 2 | P003       | Electronics | 1200       | 1500          | 5           | 1                |
| 3 | P004       | Grocery     | 5          | 7             | 300         | 45               |
| 4 | P005       | Grocery     | 8          | 12            | 250         | 30               |
| 5 | P006       | Home        | 45         | 60            | 10          | 8                |



Next steps:

[Generate code with sales\\_df](#)

[New interactive sheet](#)

```
# 2. NumPy Operations: Profitability and Projections
cost = sales_df['Cost_Price'].values
selling = sales_df['Selling_Price'].values

# Calculate Profit Margin per unit using NumPy vectorization
profit_per_unit = selling - cost
sales_df['Profit_Margin'] = profit_per_unit

# Calculate Total Potential Revenue from current stock
total_rev = np.sum(selling * sales_df['Stock_Level'].values)

print(f"Total Potential Revenue in Warehouse: ${total_rev:,.2f}")
print(f"Average Profit across all products: ${np.mean(profit_per_unit):.2f}")
```

Total Potential Revenue in Warehouse: \$27,150.00

Average Profit across all products: \$81.00

```
# 3. Pandas Filtering and Grouping
# Filter for high-value items (Price > 100)
high_value_items = sales_df[sales_df['Selling_Price'] > 100]
```



```
# Group by Category to see total daily movement
category_movement = sales_df.groupby('Category')['Daily_Units_Sold'].sum()

print("Daily Units Sold by Category:")
print(category_movement)
print("\nHigh Value Items Inventory:")
display(high_value_items)
```

Daily Units Sold by Category:

Category

Electronics 3

Grocery 75

Home 23

Name: Daily\_Units\_Sold, dtype: int64

High Value Items Inventory:

|   | Product_ID | Category    | Cost_Price | Selling_Price | Stock_Level | Daily_Units_Sold | Profit |
|---|------------|-------------|------------|---------------|-------------|------------------|--------|
| 0 | P001       | Electronics | 500        | 650           | 15          | 2                |        |
| 2 | P003       | Electronics | 1200       | 1500          | 5           | 1                |        |

Next steps:

[Generate code with high\\_value\\_items](#)

[New interactive sheet](#)

# 4. Python Logic: Inventory Restock Alert System

```
print("--- INVENTORY ALERT REPORT ---")
```

# Calculate 'Days of Stock' left

```
sales_df['Days_Left'] = sales_df['Stock_Level'] / sales_df['Daily_Units_Sold']
```

```
i = 0
```

```
while i < len(sales_df):
```

```
    item = sales_df.iloc[i]
```

```
    days = item['Days_Left']
```

```
    if days < 3:
```

```
        status = "CRITICAL: Order immediately!"
```

```
    elif days < 7:
```

```
        status = "WARNING: Low stock."
```

```
    else:
```

```
        status = "OK: Sufficient stock."
```

```
    print(f"Product {item['Product_ID']} ({item['Category']}): {status} (Ends in {days:.1
```

```
    i += 1
```

```
--- INVENTORY ALERT REPORT ---
```

```
Product P001 (Electronics): OK: Sufficient stock. (Ends in 7.5 days)
```

```
Product P002 (Home): OK: Sufficient stock. (Ends in 8.0 days)
```

```
Product P003 (Electronics): WARNING: Low stock. (Ends in 5.0 days)
```

```
Product P004 (Grocery): WARNING: Low stock. (Ends in 6.7 days)
```

```
Product P005 (Grocery): OK: Sufficient stock. (Ends in 8.3 days)
```

```
Product P006 (Home): CRITICAL: Order immediately! (Ends in 1.2 days)
```

